

7. Herramientas para la traducción e interpretación

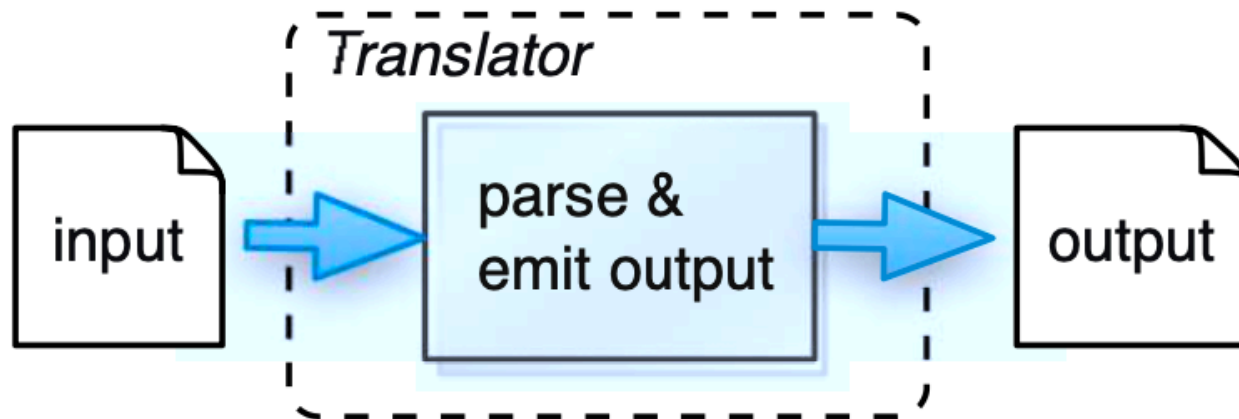
- Definición y conceptos de traducción
- Tipos de traducción
- Definición y conceptos de interpretación
- Tipos de interpretación

Definición y conceptos de traducción

- Un traductor es un programa que mapea construcciones de entrada hacia construcciones de salida.
- Los traductores varían en dificultad dependiendo de un número de factores incluyendo el orden relativo de las entradas y salidas, la presencia de referencias hacia adelante, la necesidad de preservar comentarios o formatos, etc.
- La traducción involucra el entendimiento completo de cada frase, para elegir la construcción de salida apropiada, llenando los huecos con información del modelo de entrada.

Traducción dirigida por sintaxis

- Un traductor dirigido por sintaxis lee una entrada e inmediatamente emite una salida.
- Los traductores dirigidos por sintaxis consisten de una gramática (o un analizador sintáctico equivalente) y acciones de salida.



Traducción dirigida por sintaxis

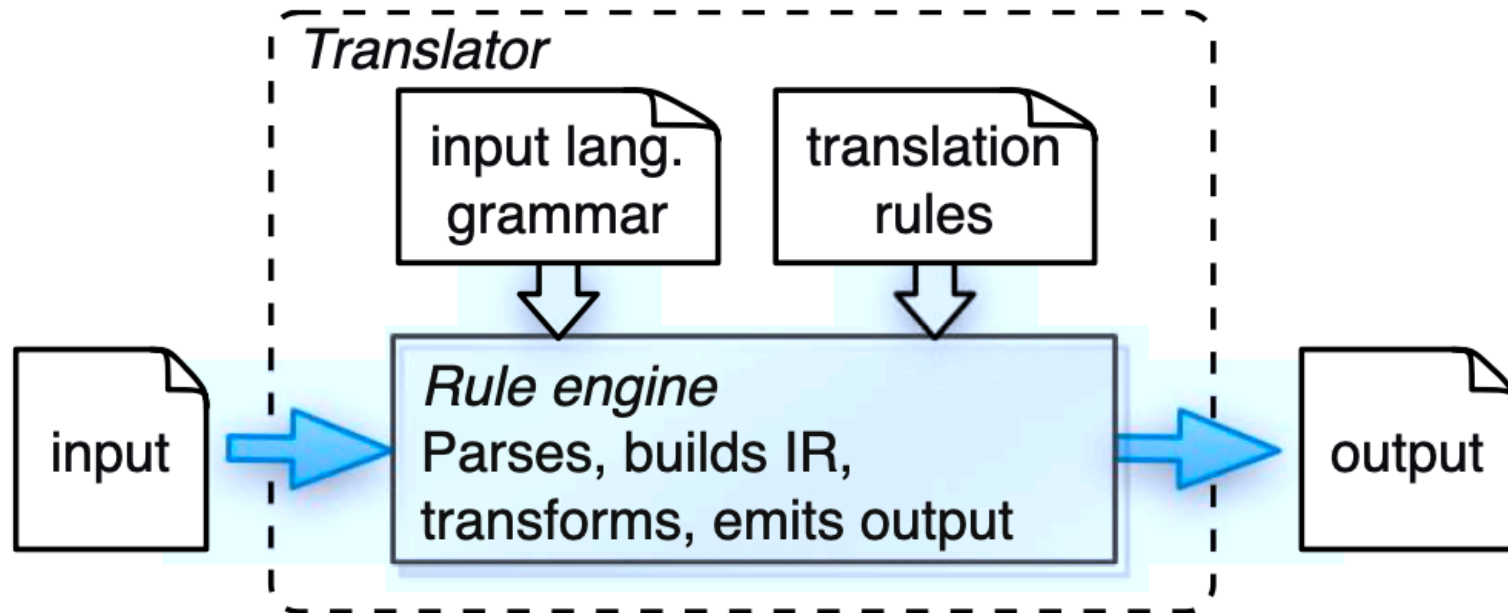
- Dado que los traductores dirigidos por sintaxis sólo dan una pasada a través de la entrada, estos no pueden lidiar con referencias hacia adelante. Tampoco pueden realizar traducciones muy sofisticadas porque sólo tienen un nivel de profundidad. No pueden analizar un modelo de entrada, lo cual se requiere para traducciones más complejas.
- Los traductores dirigidos por sintaxis tampoco funcionan cuando el orden de las construcciones de entrada y salida difieren significativamente.
- Para construir un traductor dirigido por sintaxis, se requiere escribir bastante código informal de manera manual (sin un generador).

Traducción basada en reglas

- Para construir un traductor, se debe definir un mapeo de la entrada a la salida (un conjunto de reglas “esto lleva a aquello”) sin importar la arquitectura del traductor utilizada.
- Existen varios sistemas basados en reglas como ASF+SDF, Stratego/XT, TXL y ANTLRMorph.
- Para utilizar estas herramientas, se debe alimentar la gramática de un lenguaje y un conjunto de reglas.

Traducción basada en reglas

- A partir de la gramática, la herramienta construye un árbol de sintaxis de la entrada y después aplica reglas para obtener la salida.

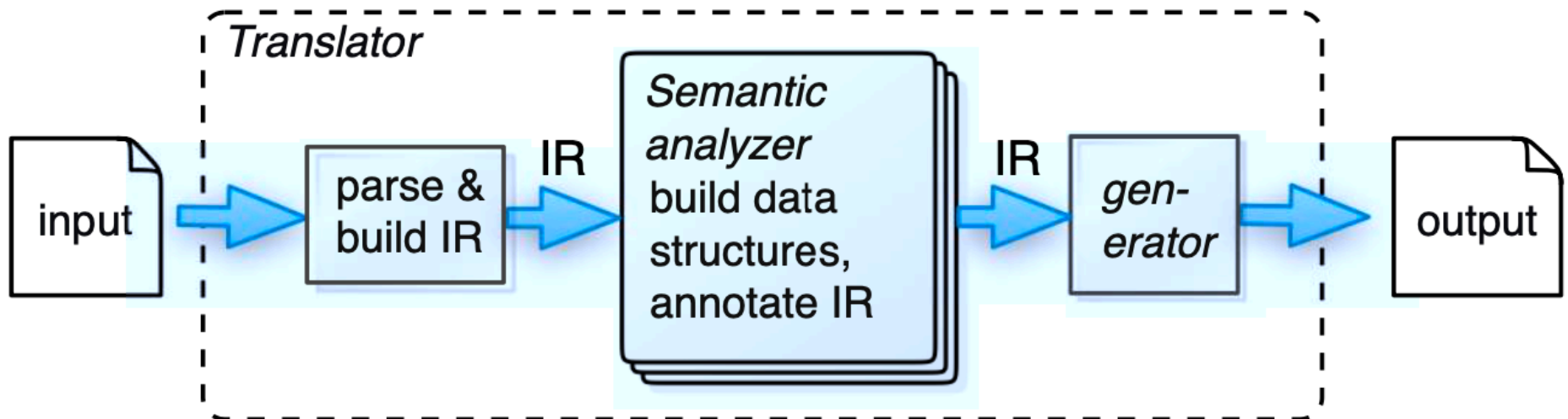


Traducción dirigida por modelo

- En un traductor dirigido por modelo, todo se centra en un modelo de entrada creado por el analizador sintáctico.
- En su forma más simple, un traductor dirigido por modelo crea un árbol de sintaxis y después lo recorre para generar la salida con enunciados de impresión (print).
- La única diferencia entre este tipo de traductor y uno dirigido por sintaxis es que se genera texto mientras se recorre un árbol en lugar de generarlo mientras se analiza un flujo de tokens.

Traducción dirigida por modelo

- La ventaja de crear un árbol de sintaxis primero es que se puede hacer análisis semántico antes de generar la salida.
- Desde un punto de vista de ingeniería de software, es deseable mantener el analizador sintáctico sin acciones específicas del lenguaje, para poder reutilizarlo para otras tareas.



Definición y conceptos de interpretación

- Para ejecutar un programa que no está escrito en lenguaje máquina, se debe interpretar el programa o traducirlo a un programa equivalente en un lenguaje que se pueda ejecutar en la máquina.
- La parte de la traducción ya fue vista anteriormente, ahora se cubrirá la interpretación de alto y bajo nivel.

Interpretación dirigida por sintaxis

- Un intérprete dirigido por sintaxis imita lo que se hace cuando se rastrea código de manera manual. Según se avanza por el código, se analiza, valida y ejecutan instrucciones. Todo se lleva a cabo en el analizador sintáctico.
- La buena noticia es que este tipo de intérprete sólo tiene dos partes:
 - El analizador sintáctico: Reconoce las construcciones de entrada y de inmediato dispara acciones.
 - El intérprete: Mantiene el estado y contiene los métodos que implementan las instrucciones.

Interpretación dirigida por sintaxis

- La mala noticia es que los intérpretes dirigidos por sintaxis son convenientes solamente para un rango pequeño de lenguajes.
- Trabajan mejor con lenguajes de dominio específico pequeño que con lenguajes de programación de propósito general.
- La interpretación de enunciados de selección, iteración, funciones y clases es extremadamente torpe en este tipo de intérpretes.
- Se recomienda utilizar un intérprete dirigido por sintaxis cuando el lenguaje de entrada parezca una secuencia de instrucciones o enunciados declarativos (lenguajes gráficos, protocolos de red, lenguajes para procesamiento de texto, lenguajes de guiones, etc.).

Interpretación basada en árbol

- Un intérprete basado en árbol es como la primera parte de un compilador, pero con un intérprete en vez del generador de código.
- Este tipo de intérprete no es muy distinto de uno dirigido por sintaxis, la diferencia es que se construye el árbol de sintaxis con el analizador léxico y después se realiza la interpretación recorriendo el árbol.

Interpretación basada en árbol

- Los cambios más significativos a partir del intérprete dirigido por sintaxis son:
- Se separa la definición de símbolos de su resolución, dividiendo las tareas en análisis y ejecución.
- Los intérpretes basados en árbol son más flexibles porque se pueden hacer sustituciones mientras se construye el árbol de sintaxis, si es necesario.

Interpretación basada en código intermedio

- Este tipo de interpretación se basa en la traducción del código fuente a un código intermedio para el cual se cuenta con un intérprete o máquina virtual.
- En este caso se utilizará un código intermedio basado en una secuencia de tuplos.

Interpretación basada en código intermedio

- La notación de tuplos se utiliza para representar a través de una secuencia de tuplos, un programa cuya sintaxis es correcta.
- La estructura general de un tuplo es la siguiente:

(número-línea, instrucción, parámetros, salto-verdadero, salto-falso)

Interpretación basada en código intermedio

- Como ejemplo se hará la traducción del siguiente programa en pseudocódigo a código intermedio.

```
inicio-programa
  leer "Cuantas calificaciones", n
  prom = 0

  i = 0
  mientras (i < n)
    leer "Da una calificacion", cal
    prom = prom + cal
    i = i + 1
  fin-mientras

  prom = prom / n

  si (prom > 5) entonces
    escribir "Aprobado con: ", prom
  fin-si

  si (prom == 10) entonces
    escribir "Excelente"
  fin-si
fin-programa
```

Interpretación basada en código intermedio

inicio-programa

leer "Cuantas calificaciones", n
 prom = 0

i = 0

mientras (i < n)

leer "Da una calificacion", cal
 prom = **prom** + cal
 i = **i** + 1

fin-mientras

prom = **prom** / n

si (**prom** > 5) **entonces**

escribir "Aprobado con: ", **prom**
 fin-si

si (**prom** == 10) **entonces**

escribir "Excelente"

fin-si

fin-programa

(1, leer, ["Cuantas calificaciones", n], 2, 2)

(2, asignacion, [prom, 0], 3, 3)

(3, asignacion, [i, 0], 4, 4)

(4, comparacion, [i, <, n], 5, 8)

(5, leer, ["Da una calificación", cal], 6, 6)

(6, asignacion, [prom, prom, +, cal], 7, 7)

(7, asignacion, [i, i, +, 1], 4, 4)

(8, asignacion, [prom, prom, /, n], 9, 9)

(9, comparacion, [prom, >, 5], 10, 11)

(10, escribir, ["Aprobado con: ", prom], 11, 11)

(11, comparacion, [prom, ==, 10], 12, 13)

(12, escribir, ["Excelente"], 13, 13)

(13, fin, [], -1, -1)

Interpretación basada en código intermedio

- Para incorporar la generación de código intermedio en el intérprete de pseudocódigo, modifique el analizador sintáctico para construir una estructura de datos (arreglo, ArrayList, etc.) que almacene los tuplos (objetos con toda la información que requiere un tuplo: número de línea, instrucción, parámetros de la instrucción, salto a línea en caso verdadero, salto a línea en caso falso).

Interpretación basada en código intermedio

```
private boolean enunciadoAsignacion() {
    int beginIndex = tokenIndex;

    if (match("VARIABLE"))
        if (match("IGUAL"))
            if (expresion()) {
                String parameters[] = new String[tokenIndex-beginIndex-1];
                parameters[0] = tokens.get(beginIndex).data;
                int i = beginIndex+2;

                while (i < tokenIndex) {
                    parameters[i-beginIndex-1] = tokens.get(i).data;
                    i++;
                }

                Tuplo t = new Tuplo(indexOfTuple,"asignacion",parameters,indexOfTuple+1,indexOfTuple+1);
                tuplos.add(t);
                indexOfTuple++;
                return true;
            }

    return false;
}
```

Interpretación basada en código intermedio

1. Se debe definir una clase que represente un objeto tipo Tuplo con todos sus datos, constructor, getters, setters, etc.
2. Se debe declarar una estructura de datos para almacenar una colección de objetos Tuplo (se puede utilizar un arreglo, pero es preferible un ArrayList para que pueda ir creciendo según se requiera). Esta estructura de datos debe crearse en el constructor del analizador sintáctico para poder utilizarse en toda la clase.
3. En cada método del analizador sintáctico, donde se verifica algún tipo de enunciado (asignación, leer, escribir, etc.) se debe agregar código que construya un tuplo con la información del enunciado y después se debe agregar el objeto Tuplo a la estructura de datos.